Museum of Science Educator Portal — Code and Design Study Guide

Project: Museum of Science (MOS) Boston educator portal — LIS 467 Assignment 3 **Company:** Agentic Curriculum Designers, Inc. **Live site:** http://web.simmons.edu/~colek/ **Written:** 2026-11-04, from the code as it exists in `assignment3/webserver/`

This guide explains every piece of code in the site, why it is written the way it is, and what each language feature does, so you can study it and answer questions about it. Where I found a bug or weak spot while reading, it is listed honestly in Part 9.

Part 1 — The big picture

1.1 What the site does

A visitor chooses an audience (Students, Parents or Educators), types a question into the "Ask LLM" box, and gets back:

1. a short AI-written answer, grounded only in the museum's own content;
2. exhibit and YES-unit cards in the left column;
3. standards and research-document links in the right sidebar.

1.2 The three layers

Layer	Files	Runs where	Job
Pages and design	`index.html`, `search-results.html`, `css/colors.css`, `css/layout.css`	Browser	Structure, look, responsive layout
Behaviour	`js/carousel.js`, `js/search.js`	Browser	Carousel, remembering the audience, running the search
Server	`ask.php`	Simmons Apache + PHP 8.4	Holds the secret API key, calls Claude, returns JSON
Data	`data/index.json` (built offline)	Downloaded by the browser, also read by PHP	206 documents, each with a 384-number embedding

1.3 Why it is split this way

- **The browser does the vector search** because the Simmons server has PHP but no Node.js, so it cannot run the embedding model. The browser can (WebAssembly, through Transformers.js).

- **The server does the AI call** because the Anthropic API key must never be sent to a browser. PHP keeps it in a file outside the web pages and only returns the answer.
- **The index is built ahead of time** on a laptop. Embedding 206 documents takes minutes; doing it per visitor would be impossible.

This split is called a **hybrid** design: the browser decides *which documents are relevant*, the server decides *what to say about them*.

1.4 Data flow for one search

```
HOME PAGE (index.html)
  carousel.js saves the chosen audience in localStorage
  the search form submits by GET
        |
        v
search-results.html?audience=educators&q=earthquake
  search.js reads the URL parameters
        |
        |-- fetch data/index.json       (206 docs + vectors)
        |-- import Transformers.js      (from the jsDelivr CDN)
        v
load the MiniLM model, embed the question  ->  384 numbers
        |
        v
dot product with every document (filtered by audience)
        |
        v
top 6 document IDs
        |
        |  POST {query, persona, doc_ids}
        v
ask.php (server)
  validates the IDs, builds a context block, calls Claude Haiku
        |
        |  {answer, persona, results[6]}   (JSON)
        v
render() in search.js: answer text + cards + sidebar links
```

If any step in the browser fails, `search.js` falls back to server-only keyword search (Section 5.6). If the server call fails, the browser still shows the vector results without an AI answer.

Part 2 — The data pipeline (built offline)

None of this runs on the web server. It produces `data/documents.json` (readable text) and `data/index.json` (the same plus vectors).

2.1 Source files (`data/sources/`)

File	Contents
YES Curricula List.md	Youth Engineering Solutions units by section, with grade bands
Exhibits ... Alignment.md	26 exhibits, the YES activity each matches, the MA STE standard, and the rationale
MA STE Standards ... Alignment.md	20 standards, their curriculum category, YES units, and descriptions
STEM Research Documents Directory.md	The 100 curated PDF links
Information_Architecture_MOS_Ed_portal.xlsx	The IA: every link, its level in the menu, and its category
`site_links.json`, `site_pages.json`	The real mos.org / yes.mos.org page for each unit, exhibit and standard, and its official description
`link_replacements.json`	A working URL for each of 78 dead research-document links
`pdf_extracts.json`	The opening text of each linked document

2.2 `fetch_site_pages.py`

Visits every mos.org / yes.mos.org URL and records its status, title, meta description and the first 1,500 characters of body text.

- `ThreadPoolExecutor(max_workers=8)` fetches 8 pages at a time instead of one after another. This is I/O-bound work (waiting on the network), so threads help.
- `BeautifulSoup` parses the HTML. It removes `script`, `style`, `nav`, `header` and `footer` tags so only content is left.
- The result is saved keyed by URL, so later scripts can look up a page's description by its URL.

2.3 `fetch_extracts.py`

Downloads each research document and extracts its opening text.

- `open_url()` tries two User-Agent strings (a browser one, then `curl/8.7.1`) because some hosts block one kind or the other. Fema.gov was one of them.
- PDFs are saved to a temp file and run through the command-line tool `pdftotext -l 3` (first three pages only).
- HTML pages are parsed with BeautifulSoup the same way as above.
- If the extracted text is under 80 characters, the status is `"no text"`. These are scanned PDFs with no machine-readable text. Two of the 99 are like this.

- Errors are caught and recorded in the result instead of crashing the whole run. That pattern ("record the failure, keep going") is what makes a 100-URL job practical.

2.4 `build_documents.py`

Reads all the sources and writes `documents.json`. Each document has `id`, `type`, `title`, `category`, `text` and `url`. The `text` field is what gets embedded, so it is where the search quality comes from.

Helper functions

Function	What it does
`strip_md()`	Removes Markdown links, bold and underline markup so text is plain
`table_rows()`	Parses a Markdown table into lists of cells (skipping the header and `---` rows)
`norm()`	Normalises a unit name: lower-case, drops bracketed text and stop words like "engineering" and "yes", so "Engineering Earthquake-Resistant Buildings" can match "Earthquake-Resistant Buildings" in another table
`matches()`	True if a unit name equals, or ends with, one of the names in an "activity" cell
`activity_names()`	Splits a cell like `Wee Engineer Units (A, B) / Engineering C` into separate normalised names
`parse_yes()`	Reads the YES list into sections, each with its units
`parse_directory()`	Reads the 100 numbered research links with their section heading
`xlsx_categories()`	Maps each PDF URL to its Level 4 category in the spreadsheet

Document types produced (206 in total)

Type	Count	How its `text` is built
`yes-unit`	53	Unit name + section + official description + grade band + matched MA STE standards + related exhibits
`yes-overview`	7	One per YES section: its description and unit list
`exhibit`	26	Alignment rationale + official mos.org description + matched YES activity (or "curriculum gap") + primary standard
`ste-standard`	20	Standard description + category + YES units + exhibits that share the standard
`ste-overview`	1	The framework objectives
`research-document`	99	Title + publisher + category + the first 700 characters of the document

The research-document step in detail: for each of the 100 directory entries it looks up a replacement URL (78 were dead), drops the one whose replacement duplicated `doc-001`, and skips any link whose extract status is not `ok` or `no text`. That leaves 99. The ID is `doc-001`, `doc-002` and so on.

Why cross-linking matters: a YES unit's text lists its aligned standards and related exhibits, and an exhibit's text lists its YES activity. That means a query about a *standard* can surface the *exhibit* too, even though the words differ, because the vector for the exhibit already "contains" the standard's meaning.

2.5 `src/embeddings.js` and `scripts/buildIndex.js` (Node.js)

```
let embedderPromise = null;
function getEmbedder() {
  if (!embedderPromise)
    embedderPromise = pipeline("feature-extraction", "Xenova/all-MiniLM-L6-v2");
  return embedderPromise;
}
export async function embedText(text) {
  const embedder = await getEmbedder();
  const output = await embedder(text, { pooling: "mean", normalize: true });
  return Array.from(output.data);
}
```

- **Lazy singleton:** the model is loaded the first time it is needed, and the same Promise is reused. This pattern avoids loading a 23 MB model 206 times.
- **`pooling: "mean"`:** the model outputs one vector per word-piece; averaging them gives one vector for the whole text.
- **`normalize: true`:** scales the 384-number vector to length 1. This is the key to the search maths (Section 5.4).
- `buildIndex.js` loops over `documents.json`, embeds `` `${doc.title}. ${doc.text}` `` for each, and writes `index.json` = each document plus an `embedding` array. The file is about 2.4 MB.

2.6 `build_nav.py` — the navigation, from the IA spreadsheet

The navigation is the only HTML that is **generated**. It reads the IA spreadsheet and rewrites the block between `<!-- NAV:START -->` and `<!-- NAV:END -->` in both pages.

- `read_tree()` finds the header row ("Level 1 ..."), then for each row walks Level 1 → Level 5, creating nested `Node` objects. `Node.child()` uses `dict.setdefault` so repeated labels reuse the same node.
- `Node.href` picks a link: one shared URL becomes the target; several URLs (a category) become `#`.
- `render_item()` / `render_list()` recursively produce nested `<ul>` lists. A menu made only of leaf links gets the class `menu-scroll` (26 exhibits is long), while a menu that contains flyouts must not scroll or the flyouts would be clipped.
- `render_nav()` forces the persona order **Students, Parents, Educators, About** (the wireframe's order, not the spreadsheet's) and relabels them "For Students" and so on.
- `inject()` does a regex replace between the two comment markers (`re.S` lets `.` match newlines). Everything outside the markers is untouched, so hand edits survive re-running the script.
- `--depth` and `--dry-run` options let you shorten the menu or preview it.

Design idea: the spreadsheet is the single source of truth for site navigation. Change the IA, re-run the script, and the menus update. This is "generate, don't hand-write" for repetitive markup.

Part 3 — Page design and HTML

3.1 From wireframe to markup

The A2 wireframes drew two pages. Each wireframe block became one semantic element:

Wireframe block	HTML	Page
Navigation bar with logo	`<header>` containing `<nav class="site-nav">`	both
Hero with "Featured Exhibit" button	`<section class="hero">`	home
Ask LLM panel with 3 audience radios	`<section class="ask-llm">` with a `<form>`	both
Upcoming Exhibits carousel and dots	`<section class="exhibits">`	home
Calendar widget and Event Info	`<section class="events">`	home
Search query, restated question and AI answer	`<section class="query">`	results
Exhibit result cards	`<article class="exhibit-result">`	results
Resource Results sidebar	`<aside class="results-side">` containing `<section class="resource-results">`	results
FAQ, social icons, map	`<footer class="site-footer">`	both

The wireframe was turned into HTML on the Linux machine. The markup was written, not generated, apart from the navigation.

3.2 Anatomy of `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Museum of Science Education Portal</title>
  <link rel="stylesheet" href="css/colors.css">
  <link rel="stylesheet" href="css/layout.css">
</head>
```

Line	Why it matters
`lang="en"`	Screen readers pick the right pronunciation; browsers offer correct translation
`charset="UTF-8"`	Lets the page show characters like the em dash and "ñ" without garbling
`meta viewport`	**The essential line for mobile.** Without it a phone pretends to be about 980 px wide and shrinks the page. This tells it to use the real device width
Two `<link>`s	`colors.css` first (defines variables), `layout.css` second (uses them). Order matters because layout uses the variables

3.3 Semantic elements used, and why

- `<header>`, `<nav aria-label="Main">`, `<main>`, `<footer>`: landmarks that screen-reader users can jump between.
- `<section>`: a themed group of content, normally with a heading.
- `<article>`: a self-contained item that makes sense on its own, so each search result is an `<article>`.
- `<aside>`: content related to, but separate from, the main content: the results sidebar.
- `<figure>` / `<figcaption>`: an image with its caption.
- `<em>` (index.html line 223) means emphasis; `<strong>` (index.html line 296, search-results.html lines 223 and 318) means importance. Both carry meaning, unlike `<i>` and `<b>`, which are only visual.

3.4 Accessibility details worth knowing

- `aria-label="Main"` on `<nav>` names the landmark.
- `aria-hidden="true"` on the dots, the calendar squares and the social icons tells screen readers to skip decorative elements.
- `<legend class="visually-hidden" hidden>I am a</legend>` names the radio group. See Part 9, item 12, for a small problem with it.
- The carousel gets `tabindex="0"`, `role="region"` and an `aria-label` from JavaScript so keyboard users can focus it and scroll with the arrow keys.
- `<label><input type="radio"> Students</label>`: wrapping the input in its label makes the whole text clickable.
- `alt` text on every image describes the exhibit.
- The `.visually-hidden` CSS class hides a heading visually but keeps it for assistive technology. It is used for the `<h2>` in the results page, which stops the heading levels from jumping from `<h1>` to `<h3>`.

3.5 The form is a plain GET request

```
<form action="search-results.html" method="get">
  ...radios name="audience"...
  <input type="search" name="q" ...>
</form>
```

Submitting navigates to `search-results.html?audience=educators&q=earthquake+proof+buildings`. The browser builds that query string from each field's `name` and value. No JavaScript is needed for the hand-off, so

it works even if the scripts fail. `search.js` then reads the same URL.

3.6 The carousel markup

Twenty-six `<figure class="card">` elements sit inside `<div class="carousel">`. Eighteen exhibits have a photo; eight use `<div class="placeholder-img">` with the exhibit name because no photo was provided. The six `<span>`s inside `.dots` are the pagination dots.

3.7 The results page has placeholder content

`search-results.html` ships with three static example articles, six "Document Title" resource rows, and lorem ipsum text. This is the wireframe, made real. When `search.js` runs, `render()` removes those placeholders and substitutes the real results. If JavaScript is disabled, the visitor sees the static example.

Part 4 — The CSS

4.1 `colors.css` — the design tokens

```
:root {
  --blue-dark: #00205C;   --blue-mid: #00318E;   --blue-bright: #0042BC;
  --gray-dark: #575757;   --gray-mid: #898989;   --gray-light: #BCBCBC;
  --purple-dark: #57007F; --purple-mid: #7900B2; --purple-bright: #9C00E5;
  --font-heading: "Cascadia Code", monospace;
  --font-body: Verdana, Geneva, sans-serif;
}
```

- `:root` is the `<html>` element. Variables defined there are available everywhere.
- A **custom property** (CSS variable) is used as `var(--blue-dark)`. Change the value in one place and every use updates. This is why the palette from the A2 mood board lives in its own file.
- The W3C CSS validator prints a warning for each use of `var()` ("CSS variables are not statically checked"). It is not an error. The validator simply cannot predict the value.
- The file also defines utility classes (`.bg-purple-dark`, `.text-blue-mid`, and so on). The pages do not use most of them yet; they are a ready-made vocabulary for Assignment 4.
- Base rules: body text is Verdana in `--gray-dark`; all headings are Cascadia Code in `--blue-dark`; links are `--blue-bright` and turn `--purple-bright` on hover.

Font fallback: `"Cascadia Code", monospace` means "use Cascadia Code if the visitor has it installed, otherwise any monospace font". The site does not download fonts, so it stays fast, but the heading font will vary by device.

4.2 `layout.css` — concept by concept

Reset

```
* { box-sizing: border-box; }
body { margin: 0; }
```

By default, `width: 100%` plus padding makes an element *wider* than 100%. `border-box` includes padding and border in the width, which makes layout arithmetic predictable. Almost every modern stylesheet starts with this.

Flexbox (one-dimensional layout)

```
.site-nav { display: flex; align-items: center; gap: 2.5rem; ... }
```

`display: flex` lays children out in a row. `align-items: center` centres them vertically. `gap` is the space between them. Flexbox is used for: the nav bar, the radios row, the carousel, the events row (`flex-wrap: wrap` lets it fold onto two lines), the footer (`justify-content: space-between` pushes the two halves apart), and the `.doc` rows.

Grid (two-dimensional layout)

```
.results-layout { display: grid; grid-template-columns: 3fr 2fr; gap: 0.75rem; }
.exhibit-result { display: grid; grid-template-columns: 1fr 1fr; }
.calendar-grid  { display: grid; grid-template-columns: repeat(6, 1fr); }
```

`fr` means "fraction of the leftover space". `3fr 2fr` gives the results column 60% and the sidebar 40%. This is the assignment's **two columns** requirement, and it appears at three levels (page, card, calendar).

The dropdown menu

```
.site-nav li { position: relative; }
.site-nav .sub { display: none; position: absolute; top: 100%; left: 0; z-index: 10; ... }
.site-nav .sub .sub { top: 0; left: 100%; }
.site-nav li:hover > .sub,
.site-nav li:focus-within > .sub { display: block; }
```

- Each `<li>` is `position: relative`, which makes it the reference point for its child.
- The submenu is `position: absolute; top: 100%` so it sits directly below its parent, without pushing other content around. `z-index: 10` keeps it above the page.
- A second-level menu is `top: 0; left: 100%`: beside its parent, forming a flyout.
- `li:hover > .sub` shows a menu on mouse hover. The `>` means "direct child only", so hovering "Students" does not open every nested menu at once.
- `:focus-within` does the same job for keyboard users (and for taps on a phone): the menu stays open while any link inside it has focus. This is a CSS-only dropdown with no JavaScript.
- `.menu-scroll { max-height: 70vh; overflow-y: auto; }` lets a long list (26 exhibits) scroll within 70% of the screen height.
- `.has-sub > a::after { content: "\25B8"; }` adds a small ▸ arrow with a pseudo-element, so no extra HTML is needed.

The carousel (no library)

```
.carousel { display: flex; gap: 1rem; overflow-x: auto; scroll-snap-type: x mandatory; }
.carousel .card { flex: 0 0 min(300px, 70%); scroll-snap-align: start; }
```

- `overflow-x: auto` makes the row scroll sideways when it is wider than the screen. On a phone this is native swipe scrolling.
- `scroll-snap-type: x mandatory` plus `scroll-snap-align: start` makes cards "click" into place.
- `flex: 0 0 min(300px, 70%)` means *do not grow, do not shrink, base width = the smaller of 300 px or 70% of the container*. `min()` lets one rule handle both wide and narrow screens.

Images

```
.card img, .exhibit-result img { display: block; width: 100%; aspect-ratio: 16 / 9; object-fit:
        cover; }
```

- `aspect-ratio: 16 / 9` reserves the right height before the image loads, which prevents the page from jumping.
- `object-fit: cover` crops rather than stretches an image that does not match the box.
- The source files are also resized to 1200×675 pixels. CSS only controls display size; shrinking the actual file is what saves bandwidth (the assignment's image-manipulation point).

Result cards

`.exhibit-result` is a two-column grid: `.info` (grey panel with title, snippet, standard, button) next to the `<figure>`. `.yes-card .yes-media` replaces the photo with a dark-blue panel with a purple `YES UNIT` badge (`.yes-badge`), because YES units have no photo. `border-radius: 999px` makes a pill shape.

Buttons

`.btn` is a link styled as a pill (`display: inline-block` so padding works, `border-radius: 999px`). `.btn-dark` flips the colours.

Visually hidden

```
.visually-hidden { position: absolute; width: 1px; height: 1px; margin: -1px; overflow: hidden;
        clip: rect(0,0,0,0); white-space: nowrap; border: 0; }
```

Shrinks an element to nothing and moves it off-flow without removing it from the accessibility tree. This is different from `display: none`, which hides it from everyone.

4.3 Responsive design

The site is **desktop-first**: the base rules are for wide screens, and two `max-width` media queries override them as the screen shrinks.

```
@media (max-width: 800px) { ... }   /* tablet */
@media (max-width: 600px) { ... }   /* phone  */
```

Screen	What changes
≤ 800 px (tablet)	`.results-layout` becomes one column (`grid-template-columns: 1fr`), so the sidebar drops under the results; the nav wraps (`flex-wrap: wrap`); heading sizes drop (2.5 → 2 rem and 2 → 1.75 rem)
≤ 600 px (phone)	The nav stacks vertically; dropdowns become static, full-width, indented panels instead of floating boxes; the carousel cards grow to 90% width, so one card nearly fills the screen; the audience radios stack; `.exhibit-result` becomes one column and `figure { order: -1 }` moves the photo above its text; the footer stacks; headings shrink again

Because later rules with the same specificity win, the media-query block is placed *after* the base rules in the file. Moving it to the top would break it.

Rem and relative units: `rem` is relative to the root font size, so layouts scale if the visitor increases their browser font size. Percentages and `fr` units adapt to the screen. Fixed pixels are used only for small decorative things (the 8 px dots, the 48 px logo height).

Two ways to test: the browser's device toolbar, and, as I did, a Playwright screenshot at 390 px wide.

Part 5 — JavaScript

5.1 Why JavaScript is loaded the way it is

```
<script src="js/carousel.js" defer></script>
```

`defer` downloads the script while the page is parsing and runs it only after the HTML is fully parsed, so `document.querySelector` always finds its elements. `index.html` loads only `carousel.js`; `search-results.html` loads `carousel.js` and `search.js`, in that order.

5.2 `carousel.js`

The file has two **IIFEs** (Immediately Invoked Function Expressions): `(function () { ... })();`. The function runs once, and its variables are private. This stops `const carousel` from leaking into other scripts.

Part A — the carousel

```
const carousel = document.querySelector(".carousel");
if (!carousel) return;
```

The guard is important: `carousel.js` also loads on the results page, which has no carousel. Returning early avoids errors.

```
function updateDots() {
  const idx = Math.round(carousel.scrollLeft / carousel.scrollWidth * dots.length);
  dots.forEach((d, i) => d.classList.toggle("active", i === idx));
}
```

- `scrollLeft` is how far the row has scrolled; `scrollWidth` is the total width of all 26 cards. Their ratio times 6 gives which of the 6 dots to highlight. Each dot therefore stands for a *region* of the strip, not a single card.
- `classList.toggle("active", condition)` adds the class when the condition is true and removes it when false. One line does the whole highlight.
- `carousel.addEventListener("scroll", updateDots, { passive: true })`: `passive: true` promises the handler will not call `preventDefault()`, so the browser can keep scrolling smooth.

```
dot.addEventListener("click", () => {
  const target = (carousel.scrollWidth / dots.length) * i;
  carousel.scrollTo({ left: target, behavior: "smooth" });
});
```

Clicking dot *i* scrolls to the start of region *i*, animated.

Keyboard support: `ArrowRight` / `ArrowLeft` call `scrollBy` by one card width plus the 16 px gap.

Part B — remembering the audience

```
const KEY = "mos-audience";
const saved = localStorage.getItem(KEY);
if (saved) document.querySelectorAll(`input[name="audience"][value="${saved}"]`)
                   .forEach((r) => { r.checked = true; });
radio.addEventListener("change", () => localStorage.setItem(KEY, radio.value));
```

`localStorage` stores small strings in the browser, per site, forever. When the visitor picks "Educators" on the home page, it is saved; any page that loads later pre-selects it. The template literal `` `...${saved}...` `` builds a CSS attribute selector.

Part C — form validation

```
form.addEventListener("submit", (e) => {
  const q = form.querySelector('input[name="q"]');
  if (q && !q.value.trim()) {
    e.preventDefault();
    q.focus();
    q.setCustomValidity("Please enter a question.");
    q.reportValidity();
  } else if (q) { q.setCustomValidity(""); }
});
```

`e.preventDefault()` cancels the navigation. `setCustomValidity` + `reportValidity()` shows the browser's native speech-bubble error. Clearing the custom message afterwards is essential, or the field would stay invalid forever.

5.3 `search.js` — the search engine

Constants and state

```
const params   = new URLSearchParams(location.search);
const query    = (params.get("q") || "").trim();
const audience = params.get("audience") || localStorage.getItem("mos-audience") || "educators";
```

`URLSearchParams` parses `?audience=...&q=...` for you. The audience falls back from the URL, to localStorage, to "educators". The `||` operator returns the first non-empty value.

```
const PERSONA_TYPES = {
  students:  ["exhibit", "yes-unit", "yes-overview"],
  parents:   ["exhibit"],
  educators: null,
};
```

The persona filter: students never see standards or research PDFs; parents see only exhibits; `null` means no filter. PHP has the same table (enforced a second time on the server).

Safety helpers

```
function esc(str) { return
        str.replace(/&/g,"&amp;").replace(/</g,"&lt;").replace(/>/g,"&gt;").replace(/"/g,"&quot;");
        }
```

Every piece of data that goes inside an HTML string passes through `esc()`. Without it, a document title containing `<script>` would run as code. This is called **XSS (cross-site scripting)** prevention.

```
function mdToHtml(text) {
  return esc(text)
    .replace(/\*\*([^*]+)\*\*/g, "<strong>$1</strong>")
    .replace(/\*([^*]+)\*/g,     "<em>$1</em>");
}
```

The order matters: **escape first, then add tags**. If you converted markdown first and escaped afterwards, the `<strong>` tags would be escaped too. The regexes turn `**bold**` and `*italic*` into real tags. This was added because the model sometimes replied with markdown asterisks, which showed as literal `**`. The server prompt now asks for plain prose, and this function is the safety net.

Building cards

`buildExhibitCard(r)` returns an HTML string using a **template literal** (back-ticks with `${...}`).

- For an `exhibit`, it works out the image path with `exhibitImgSrc(url)`: take the last segment of the exhibit's mos.org URL (strip any `?query` / `#hash` and trailing slashes). `https://www.mos.org/visit/exhibits/exploring-ai-making-invisible-visible` → `images/exhibits/exploring-ai-making-invisible-visible.jpg`. **This works because the photos were saved under the URL slug** (a naming convention, not a lookup table). Exhibits without a photo fall back to the grey placeholder via `onerror`. *(An earlier version slugified the title instead, which matched only 6 of 18 photos; the URL slug matches all 18.)*
- The image has `onerror="this.closest('figure').innerHTML='<div class=placeholder-img>...</div>'"`, so a missing photo is replaced by a grey placeholder instead of a broken-image icon. This is how exhibits without photos still look fine.
- For a YES unit or overview it builds the blue `.yes-media` panel with a badge ("YES Unit" or "YES Overview") instead.
- `target="_blank" rel="noopener"` opens external links in a new tab; `noopener` stops the new page from controlling the old one.

`buildDocCard(r)` builds the sidebar row: a "PDF" icon box for research documents (or an emoji from `typeIcon()` for other types), a linked title and a snippet.

`render(data)`

```
const exhibits = results.filter((r) => r.type === "exhibit" || r.type === "yes-unit");
const docs     = results.filter((r) => !exhibits.includes(r));
```

Exhibits and YES units go in the main column; everything else (standards, overviews, research documents) goes in the sidebar. Then:

- the old static `<article>`s are removed (`querySelectorAll(...).forEach(el => el.remove())`);
- new cards are built; each string is turned into a DOM node via a throw-away `<div>` and `innerHTML`, collected into a **DocumentFragment**, and appended once. Appending a fragment causes a single page re-layout instead of one per card;
- if a group is empty, a friendly "No ... results" message is added;
- the AI answer is placed in the paragraph with `innerHTML = mdToHtml(answer)`.

5.4 The vector maths

```
function dot(a, b) { let s = 0; for (let i = 0; i < a.length; i++) s += a[i] * b[i]; return s; }
```

Each text is turned into a point in 384-dimensional space, and texts with similar meaning end up pointing the same way. The **cosine similarity** of two vectors measures the angle between them. When both vectors have length 1 (which `normalize: true` guarantees), cosine similarity equals the **dot product**: multiply matching numbers and add them up. The result is between –1 and 1; larger means more similar.

```
function vectorSearch(queryVec, docs, persona) {
  const allowed = PERSONA_TYPES[persona] || null;
  return docs
    .filter((d) => allowed === null || allowed.includes(d.type))
    .map((d) => ({ doc: d, score: dot(queryVec, d.embedding) }))
    .sort((a, b) => b.score - a.score)
    .slice(0, TOP_K);
}
```

A four-step pipeline: **filter** by audience → **map** each document to a score → **sort** highest first → **slice** the top 6. 206 documents × 384 numbers is about 80,000 multiplications, which is trivial for a browser.

Why this beats keyword search: a search for "earthquake proof buildings for kids" returns "Engineering Earthquake-Resistant Buildings" even though "proof" and "resistant" are different words. The model learned during training that they mean similar things. That is what "semantic" means.

5.5 `runSearch()` — the main flow

```
[indexDocs, lib] = await Promise.all([
  fetch(INDEX_URL).then((r) => { if (!r.ok) throw new Error(...); return r.json(); }),
  import(TRANSFORMERS_URL),
]);
```

- `async` / `await` lets asynchronous code read top-to-bottom. `await` pauses *this function* (not the whole page) until a Promise resolves.
- `Promise.all([a, b])` runs both jobs **at the same time** and waits for both. Downloading the 2.4 MB index and the Transformers.js library in parallel saves time.
- `import(url)` is a **dynamic import**: it loads a JavaScript module from a URL at runtime. That is how the library comes from the jsDelivr CDN without a build step. It works in a plain (non-module) script.
- `fetch(...).then(...)` is the older Promise style; `await` is the newer one. Both appear in this file.
- `lib.env.allowLocalModels = false;` tells the library to download the model from Hugging Face instead of looking on the local disk.

```
embedder = await lib.pipeline("feature-extraction", MODEL, { progress_callback: ... });
```

Downloads and starts `Xenova/all-MiniLM-L6-v2` (about 23 MB, runs on WebAssembly). `progress_callback` updates the status text ("Loading model: model.onnx 42%"). After the first visit the browser may serve it from cache, but see Part 9, item 8.

```
const output = await embedder(query, { pooling: "mean", normalize: true });
const queryVec = Array.from(output.data);
```

Critical detail: the browser must use *exactly* the same settings (mean pooling, normalised) as the Node script that built `index.json`. If they differed, the numbers would not be comparable. I checked that vectors built in Node and in the browser match with a similarity of 1.0000.

```
body: JSON.stringify({ query, persona: audience, doc_ids }),
```

Only the *IDs* of the top 6 documents are sent, not the whole documents or vectors. The server already has the data.

5.6 Fallbacks (graceful degradation)

If this fails	What happens
Fetching `index.json` or the library (network, CDN blocked)	`serverFallback()`: POST just `{query, persona}`; PHP does keyword search and the AI answer
Loading the model	Same: `serverFallback()`
The POST to `ask.php` (server down, key missing)	Shows the six vector results with the message "(AI answer unavailable — showing search results only)"
`serverFallback()` also fails	`renderError("Search failed: ...")`
Empty query	"No query provided" message

A site that has several independent ways to produce *some* answer is more robust than a site with one perfect path.

5.7 Scope note

`search.js` is a classic script, so every top-level `const` and `function` is global. That is acceptable for a one-page-per-script site but would collide if two scripts used the same name. Wrapping it in an IIFE like `carousel.js`, or converting to an ES module, would fix that.

Part 6 — The server: `ask.php`

6.1 The request in one picture

```
POST ask.php   { "query": "...", "persona": "educators", "doc_ids": ["exh-004", ...] }
  -> checks the request, loads the key and index, picks documents, asks Claude
  <- { "answer": "...", "persona": "...", "results": [
{id,title,category,type,url,score,snippet} x6 ] }
```

6.2 Section by section

Section	What it does
Headers	`Content-Type: application/json` tells the browser what is coming. `Access-Control-Allow-*` are CORS headers; an `OPTIONS` pre-flight request is answered with 204 and no body.
Config	`define()` constants for the key path, index path, model name, `TOP_K = 6` and `MAX_TOKENS = 350`. `MAX_TOKENS` caps the length (and cost) of the answer.
`PERSONA_TYPES`	Same table as JavaScript. The server re-checks it, so a tampered browser cannot request types it should not see.
`json_error()`	Sends an HTTP status code and a JSON error, then `exit`s.
Parse request	`file_get_contents('php://input')` reads the raw POST body. PHP's normal `$_POST` is empty for JSON requests, so you read the raw stream and `json_decode` it. `??` is the null-coalescing operator: "use this, or the default if missing". An unknown persona silently becomes `educators`.
Load key	Reads `/users/c/colek/.anthropic_key` (one level *above* the web folder, so it cannot be downloaded by URL), trims it, and checks that it starts with `sk-`.
Load index	`json_decode(..., true)` turns the JSON file into PHP arrays.
Retrieval	Two modes (below).
Context	Builds a numbered text block: type, title, category, the first 400 characters of text, and the link.
System prompt	A PHP heredoc (`<<<SYSTEM ... SYSTEM`), which is a multi-line string. It tells Claude to answer in 2–4 sentences, cite only the given titles, end with one suggestion, never invent resources, and use plain prose with no markdown.
Claude call	cURL: POST JSON to `https://api.anthropic.com/v1/messages` with headers `x-api-key` and `anthropic-version`. SSL verification is on. 30-second timeout.
Response	If cURL or HTTP failed, a readable placeholder answer is used. Otherwise it reads `content[0].text`. The results array is rebuilt with only the fields the browser needs, plus a 200-character `snippet`.

6.3 The two retrieval modes

Hybrid mode (when `doc_ids` is present):

```
foreach (array_slice($doc_ids, 0, TOP_K * 2) as $id) {
    if (!preg_match('/^(yes|exh|ste|doc)-\d{3}$/', $id)) continue;
    if (!isset($id_map[$id])) continue;
    if ($allowed !== null && !in_array($doc['type'], $allowed, true)) continue;
    $top[] = ['doc' => $doc, 'score' => 1.0];
    if (count($top) >= TOP_K) break;
}
```

Treat anything from the browser as untrusted: take at most 12 IDs, require each to match a strict pattern, exist in the index, and be an allowed type. The browser's ranking order is kept. The score is a placeholder `1.0` because PHP did not compute it.

Keyword fallback (no `doc_ids`):

- Split the query into words, lower-case.
- For each allowed document, count how many query words longer than 2 letters appear in its title, category and text (`str_contains`).
- Score = hits ÷ number of terms; sort high to low (`usort` with an arrow function and the spaceship operator `<=>`); keep the top 6.
- If nothing matched at all, return the first 6 allowed documents so the page is never empty.

This is crude: "building" will not match "buildings" semantically, only by substring. That is why the vector path is the primary one.

6.4 Why a server step at all?

- **Secrecy:** a key in JavaScript is visible to anyone using View Source. A key in a PHP file that is never output stays on the server.
- **RAG (retrieval-augmented generation):** the model is *only* shown six retrieved documents and told to use only those. It cannot cite a resource that is not in the index, which sharply reduces made-up answers.
- **Cost:** Claude Haiku 4.5 costs a fraction of a cent per search (roughly $0.0003).

6.5 Deployment facts you may be asked about

- Simmons serves plain HTTP only from this network, and runs Apache with PHP 8.4. There is no Node.
- Apache's `$_SERVER['HOME']` is `/usr/share/httpd`, not your home folder, so the key path is hard-coded.
- The key file needed `chmod 604` so that the Apache user could read it (see Part 9, item 6).
- The index is read from `__DIR__ . '/data/index.json'`, meaning "the data folder next to this script".

Part 7 — Following one search end to end

Query: **"earthquake proof buildings for kids"**, audience **students**.

1. On the home page the student radio is saved into `localStorage` (carousel.js, Part B). The form submits (carousel.js checks the box is not empty) to `search-results.html?audience=students&q=earthquake+proof+buildings+for+kids`.
2. `search-results.html` loads, carousel.js restores the radio, and search.js runs. `query` and `audience` come from the URL. Both text inputs are filled with the query.
3. The status line reads "⏳ Loading search index…". `Promise.all` fetches `index.json` and imports Transformers.js in parallel.
4. The model loads, with progress shown. The query is embedded to 384 numbers.
5. `vectorSearch` keeps only `exhibit`, `yes-unit` and `yes-overview` documents (students), scores them, and keeps the top 6. The first is "Engineering Earthquake-Resistant Buildings".
6. The six IDs go to `ask.php`. PHP validates them, builds the context text, and calls Claude with the persona "students".
7. Claude returns 2–4 sentences. PHP returns JSON.
8. `render()` removes the placeholders, shows the answer, builds the cards (blue YES cards for units, photo cards for exhibits) and the sidebar.

Part 8 — Design decisions and trade-offs

Decision	Reason	Cost
Browser-side embeddings	Simmons has no Node	First visit downloads about 23 MB model and 2.4 MB index
Server-side Claude call	Keeps the key secret	Needs PHP, and spending is tied to your account
Index built offline	Fast, repeatable	Must be rebuilt when documents change
Pure CSS carousel and dropdowns	No libraries; fast; accessible	Dots are approximate (6 dots for 26 cards)
Colours as CSS variables	One-place palette changes	Validator warnings
Persona filter in both JS and PHP	Defence in depth	Two copies to keep in sync
Static placeholder content kept in the HTML	Matches the wireframe; works without JS	Placeholders show briefly before results load
Replace 78 dead links	Every search result opens	The links differ from the spreadsheet's originals
Desktop-first media queries	Wireframes were designed for desktop	Mobile styles are overrides, which is more CSS than mobile-first

Part 9 — Known issues and honest caveats

I found these while reading the code for this guide. None stops the site from working, but you should know about them before you are asked about them.

1. **CSS selector typo.** In the 600 px block, `.ask-llm.compact .fieldset { flex-direction: column; }` targets a class named `fieldset`, but the element is a `<fieldset>` tag. It should be `.ask-llm.compact fieldset`. The radios on the results page still stack because the earlier rule `.ask-llm fieldset` already does it, so you do not see the effect.
2. **"Caretakers" does not match a persona.** The results-page form has a third radio with value `caretakers`, from the wireframe label. The code only knows `students`, `parents` and `educators`. Choosing Caretakers searches *all* document types (JavaScript falls back to no filter; PHP falls back to educators). The home page says "Parents" and the results page says "Caretakers"; they should match.
3. **First carousel dot is always purple.** The rule `.dots span:first-child` colours it unconditionally, so after scrolling it stays purple alongside the active dot.
4. **YES cards on a phone.** `figure { order: -1 }` puts photos above text, but YES cards use a `div`, not a `figure`, so their blue panel stays below the text. It still looks acceptable.
5. **Stale comments in `ask.php`.** The header says the key is in `~/.anthropic_key` and the index in `../data/index.json`, and that it accepts only `query` and `persona`. In reality the key is at `/users/c/colek/.anthropic_key`, the index is `data/index.json`, and it also accepts `doc_ids`.
6. **The key file is readable by other accounts.** `chmod 604` means the owner can read/write and *everyone else can read*. On a shared university server another user could in principle read it. It was needed to let Apache read the file. Keep the Anthropic spending cap low (a few dollars) and delete or rotate the key after the course.
7. **`ask.php` can be called by anyone.** `Access-Control-Allow-Origin: *` and no rate limiting mean someone who finds the URL could run up your Claude usage. The `MAX_TOKENS` cap and a spending limit are the only protection. A production site would add rate limiting and restrict the allowed origin.
8. **The model is not "kept loaded" between searches.** Each search is a full page load, so the model is re-initialised every time. Whether it must be *re-downloaded* depends on browser caching, and the Cache Storage API that Transformers.js relies on needs a secure context (HTTPS or localhost). The site is served over plain HTTP, so caching may not work. I have not measured this. Using HTTPS if Simmons allows it, or turning the results into a single page that does not reload, would keep the model in memory.
9. **`ask.php` parses the whole 2.4 MB index per request,** including the vectors it does not use. A slimmer copy without embeddings would be faster.
10. **`search.js` has global variables** (Section 5.7).
11. **Exhibit images depend on a naming convention.** If an exhibit's mos.org URL changes, its image will silently fall back to the grey placeholder. (Fixed once already: the code originally slugified the *title*, which matched only 6 of 18 photos; it now uses the URL slug.)
12. **`<legend class="visually-hidden" hidden>`.** The `hidden` attribute is display:none, so the legend is hidden from screen readers too, making the `visually-hidden` class pointless. Removing `hidden` (and keeping the class) would name the radio group for assistive technology.
13. **Placeholder content is visible to visitors without JavaScript** (the lorem ipsum cards).
14. **Hybrid-mode score is always 1.0** in the PHP response. The real score exists only in the browser.
15. **Documentation drift.** `DOCUMENTATION.md` still describes the pages as having "no JavaScript yet" and lists 260 documents. This guide supersedes it.

Part 10 — Concept glossary

Term	Meaning
Semantic search	Finding by meaning, not by matching words
Embedding	A list of numbers (here 384) representing the meaning of a text
MiniLM (all-MiniLM-L6-v2)	A small sentence-embedding model
Transformers.js	A library that runs such models in the browser
Cosine similarity / dot product	The score for how close two embeddings are
RAG	Retrieval-augmented generation: retrieve relevant documents, then have an LLM answer using only those
LLM	Large language model; here Claude Haiku 4.5
Persona	The audience (student, parent, educator) that decides what is shown
CDN	Content delivery network; a public server that hosts libraries (jsDelivr)
CORS	Browser rules about which origins may call a server
XSS	Injecting script into a page; prevented by escaping
IIFE	Function that runs immediately and keeps its variables private
Promise / async / await	Tools for code that waits for something (network, files)
Flexbox / Grid	One-dimensional and two-dimensional CSS layout systems
Media query	A CSS rule that applies only at certain screen sizes
Viewport meta tag	Tells phones to use the real screen width
Semantic HTML	Elements chosen for meaning (`article`, `nav`) rather than looks
Graceful degradation	Still providing a useful result when part of the system fails

Part 11 — Self-test questions

1. Why does the search run in the browser, while the AI answer is made on the server?
2. What would break if the browser used `normalize: false` but the index was built with `normalize: true`?
3. Why must `esc()` run before the markdown-to-HTML replacements?
4. What does `Promise.all` buy you in `runSearch()`?
5. What is the difference between `display: none` and the `.visually-hidden` class?
6. How does the CSS-only dropdown work on both hover and keyboard focus?
7. Why is `max-width: 600px` placed *after* the base rules in `layout.css`?
8. What do `scroll-snap-type` and `flex: 0 0 min(300px, 70%)` do on the carousel?
9. Why does the viewport meta tag matter?
10. What attack does the `/^(yes|exh|ste|doc)-\d{3}$/` check help prevent, and why is the persona filter repeated in PHP?
11. What happens if the Transformers.js CDN is unreachable?
12. Name two reasons an exhibit card might show a grey placeholder instead of a photo.
13. Which parts of the navigation are generated, and how do you regenerate them?
14. Why does a query about a *standard* sometimes return an *exhibit*?

Short answers. (1) Simmons has no Node, so the browser runs the model; PHP keeps the API key secret. (2) Dot product would no longer equal cosine similarity, so rankings would be skewed by vector length. (3) Otherwise the generated tags would be escaped and shown as text. (4) Downloads the index and library at the same time. (5) The first removes the element from layout and from screen readers; the second hides it visually only. (6) `li:hover > .sub` and `li:focus-within > .sub` both set `display: block`. (7) Equal specificity resolves in favour of later rules. (8) Cards click into position as you scroll, and each card is the smaller of 300 px or 70% of the width. (9) Without it phones render at about 980 px wide and shrink the page. (10) Tampered IDs; the browser is not trusted. (11) `serverFallback()` runs keyword search on the server. (12) No image file named after the URL's last segment, or it failed to load (`onerror`). (13) The navigation between the NAV markers; run `python3 scripts/build_nav.py`. (14) The exhibit's embedded text lists its matched standard and YES activity.